

Topics :

- Recursion
- Sets
- Structural Induction
- Lists
- Tuples
- Sequences

2

Recursive definition of functions

- Defines the function in terms of itself.

- **Example 1: Factorial**

- $n!$ or $\text{fac } n$, denotes the factorial of n
- $n! = \text{fac } n = n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$

- **Specification for $\text{fac } n$**

$$\text{fac } (n) = \begin{cases} 1, & \text{if } n = 0 \\ n * \text{fac } (n - 1), & \text{if } n > 0 \end{cases}$$

3

3

$\text{fac } n$ in Haskell

$\text{fac} :: \text{integer} \rightarrow \text{integer}$ -- takes an integer, returns integer

$\text{fac } n$

| $n == 0$ = 1
| otherwise = $n \times \text{fac } (n - 1)$

4

4

Structure of recursive definitions

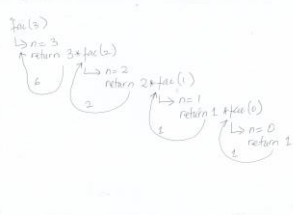
- Has the following structure:

- **Base case** - defines the function for the first input values.
- **Recursive step** - defines the function for other input values in terms of itself.
- **fac function**
 - **Base case:** $\text{fac } 0 = 1$
 - **Recursive step:** $\text{fac } n = n * \text{fac } (n - 1)$

5

5

Dry running recursive function: *fac* 3



6

Recursive definition of function *sumNumbers* n

- *sumNumbers* n computes sum of all positive whole numbers up to and including n
- Specification of *sumNumbers* n

$$\text{sumNumbers}(n) = \begin{cases} 1, & \text{if } n = 1 \\ n + \text{sumNumbers}(n-1), & n \geq 1 \end{cases}$$

Base case: $\text{sumNumbers}(1) = 1$

Recursive Step: $\text{sumNumbers}(n) = n + \text{sumNumbers}(n-1)$

7

Recursive definition of function *sumNumbers* n

sumNumbers :: Integer -> Integer

sumNumbers n

| n == 1 = 1

| otherwise = n + *sumNumbers*(n - 1)

8

Handling Errors

- Factorial is only defined for positive integers
- *fac* (-1) should return short informative error messages.
- Example: *fac* (-4)
 - $\Rightarrow (-4) \times \text{fac}(-5)$
 - $\Rightarrow (-4) \times (-5) \times \text{fac}(-6)$
 - $\Rightarrow \dots$
 - $\Rightarrow$ This is an infinite loop

9

Improved factorial function with error handling

```
fac :: Integer -> Integer
```

```
fac n
```

```
| n < 0 = error "fac only defined for positive integers"
```

```
| n >= 0 && n <= 1 = 1
```

```
| otherwise = n x fac ( n - 1 )
```

10

10

Exercise: Recursive definition of rangeSum

• rangeSum - **takes two integers and returns the sum of all integers from the first to the second, inclusive.**

• E.g. rangeSum 3 5

1. Provide a specification for this function, take care of rangeSum b a, where b > a. example rangeSum 9 6
2. Dry run rangeSum 3 5 using the notation introduction in previous slide.
3. Provide a Haskell implementation for rangeSum.

11

11

1. Specification of rangeSum

$$\text{rangeSum}(m, n) = \left\{ \begin{array}{l} \end{array} \right.$$

12

12

2. Dry running: rangeSum 3 5

13

13

3. Provide implementation for: *rangeSum*

14

14

Fibonacci sequence F_n

- Specification for fib (n)

$$\text{fib}(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2), & \text{otherwise} \end{cases}$$

15

15

Fibonacci sequence F_n

- **Base case:** $F_0 = 0, F_1 = 1$
- **Recursive step:** $F_n = F_{(n-1)} + F_{(n-2)}$ - This computes the nth Fibonacci number in the series.
- **Any integer in the sequence is the sum of the previous 2 integers.**
- 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ... and so on

16

16

Exercise - Dry run: F_5

17

17

fib :: Integer -> Integer

```
fib (n)
| n == 0      = 0
| n == 1      = 1
| n < 0       = error "n should not be negative"
| otherwise   = fib (n - 1) + fib (n - 2)
```

18

Exercise - Take home

- Give a recursive definition of a function **sumFac** defined as follows:

```
=> sumFac :: Integer -> Integer
=> sumFac n = fac 0 + fac 1 + ... + fac (n - 1) + fac (n)
```

19

Basic Collections

To be covered - Sets, Lists, Tuples, Sequences, Bags

Already covered: Basic Data Types - Boolean, Char, Integer, Float, String

20

Sets (Recap)

- A set is an unordered collection of objects
 - Order of elements does not matter
 - Repeating elements insignificant
- How to define elements of a set:
 1. List all elements of the set OR
 2. List elements such that it is clear what the missing elements are OR
 3. Give a property that characterizes the elements

21

20

21

Sets - A Recap

- $V = \{a, e, i, o, u\}$
- $S = \{1, 3, 5, 7, \dots\}$
- $B = \{n: \mathbb{Z} \mid n > 0 \wedge n \bmod 2 = 0\}$

22

22

Set - Recap - Set operations

- Assume A and B are sets
 - $A \subseteq B$
 - $A' = A$ complement
 - $A \cup B = \{x \mid x \in A \vee x \in B\}$
 - $A \cap B = \{x \mid x \in A \wedge x \in B\}$
 - $A - B = \{x \mid x \in A \wedge x \notin B\}$

23

23

Defining sets recursively

- Just like recursive definition of functions, recursive definition of sets has:
 - **Base case:** specifies initial collection of elements of the set
 - **Recursive step:** gives rules for forming new elements of the set from elements already in the set.
 - **Exclusion rule:** No elements are in the set other than those specified in the base and step

24

24

Example - A set recursively defined

- Base:** $3 \in S$
Step: if $x \in S$ and $y \in S$, then $x + y \in S$
- $\Rightarrow 3 \in S$ from base
 $\Rightarrow 6 \in S$ from step and $x = 3$ and $y = 3$
 $\Rightarrow 9 \in S$ from step and $x = 3$ and $y = 6$
 $\Rightarrow 12 \in S$ from step and $x = 9$ and $y = 3$

25

25

Structural Induction

- Properties of sets that have been defined recursively can be proved using a form of mathematical induction called structural induction.
- Recall the structure of Mathematical induction:
 - ⇒ It has a basis step
 - ⇒ It has an inductive hypothesis
 - ⇒ It has a recursive step

26

26

Example

- Consider the recursive definition:
 - Base: $3 \in S$
 - Step: if $x \in S$ and $y \in S$, then $x + y \in S$

Prove using structural induction that S is a set of all positive integers that are divisible by 3, i.e. $3n \in S$

27

27

Solution

- **Basis** - Since $3 = 3 * 1$, $3 \in S$, hence the basis case holds.
- **Inductive hypothesis**: we assume that $p(k)$ is true. $p(k)$ is $x \in S$ and $y \in S$, that is $x = 3k$ and $y = 3t$, for some integers k and t .
- Inductive step: we show that if $p(k)$ then $p(k+1)$
- $P(k+1)$ is: $x + y \in S$. So we have to show that $x + y$ can be written in the form: $3z$ for some integer z

$$x + y = 3t + 3k = 3(t + k) = 3z, \text{ where } z = t + k$$

28

28

Lists

- A list stores elements of the same type.
- Elements are enclosed in square brackets
- $[T]$ specifies a list containing elements of type T .
- $[]$ is empty list of length 0
- "singleton list" or simply "singleton" - is a list of length 1

29

29

Lists - examples

- `[2,4,50] :: [Integer]` -- list of integers
- `['2','4','2','r'] :: [Char]` -- list of characters
- `["2","int","50"] :: [String]` - list of strings
- `ex :: [Integer]` -- a list of integers
- `ex = 5 : [1, 2]` -- ex is a list with 5 at the front of the list.
- Operator `(:)` adds an element at the beginning of a list

30

30

Lists

- **A list of lists**
 - `[ ['2','S'], ['4','2','r'] ] :: [[Char]]` - **A list of list of chars**
 - `[ "2", "int", "50" ] :: [String]` - **a list of strings**
 - `[ fac, fac ]` - **a list of functions**
- **Order of elements significant**
- **How many times an element repeats also significant**

31

31

Lists - Syntax

- `[ n ... m ] =>` means `[ n, n+1, ..., m ]`
If the step size does not allow us to reach m exactly, then the last element in the list is the sequence that is closest to m, even if it appears to overshoot the limit.
- `[ 3.1 .. 7.0 ] =>` means `[ 3.1, 4.1, 5.1, 6.1, 7.1 ]`

32

32

Exercises

1. **length** Gives the number of elements in a list
2. **++** concatenates a list to the other list
3. **occurs** gives the number of occurrences of a given element in a list
 - Give the specification for each
 - Implement each using Haskell

33

33

Exercise

• specify

- $isIn(e, lst)$ that determines if the element e is in the list lst .
- Assume e is an integer, and lst is a list of integers
 $isIn :: Int \rightarrow [Int] \rightarrow Bool$
- It takes an Int and a list of Int and returns a Boolean
- We will make use of:
 - $head(lst)$ - returns an element that is head of a list
 - $tail(lst)$ - returns a list without the head of a list
 - $length(lst)$ - gives the number of elements in a list

34

34

head, tail functions

- $head[6, 3, 2, 4] \Rightarrow$ will return 6
 - $tail[6, 3, 2, 4] \Rightarrow$ will return [3, 2, 4]
 - $head[4, 3, 2, 2] \Rightarrow$ will return 4
 - $tail[3] \Rightarrow$ will return []
 - $head[[8, 7], [6, 1], [4, 6]] \Rightarrow$ will return [8,7]
- We want to use these functions to specify $isIn(e, lst)$

35

35

Exercises

1. Specify $isIn(e, lst)$
2. Dry run $isIn$
3. In the Lab, implement $isIn(e, lst)$

36

36

Tuples

- A tuple is a finite sequence of possibly different types
- Components are enclosed in round brackets
- $(T_1, T_2, \dots, T_n)$ denotes the type of all tuples whose i^{th} components have type T_i , for all i in $[1 .. n]$
- Example:

```
(50, "csi243", "2013/14") :: (Integer, String, String)
('Z', False) :: (Char, Bool)
```

37

37

Tuples

- **arity** gives the number of components in a tuple
- the empty tuple `()` is of arity 0
- the pairs is of arity 2
- tuples of arity 1 not allowed

38

38

Tuples

- **No restrictions on the types of the components of a tuple, may include:-**
 - tuples of tuples
 - Tuples of lists
 - Lists of tuples
- **Examples:-**
 - `(50, ("cs", False)) :: (Integer, (String, Bool))` - tuple of tuples
 - `([ '2', 'a', 'x'], [False, False, False]) :: ([ Char ], [Bool])` - tuple of lists
 - `[ ("cs", False), ("sc", True) ] :: [ (String, Bool) ]` - list of tuples

39

39

Tuples - Exercises

- **Give a simple specification of :**
 - `fst` which gives the first element of a tuple of arity 2
 - `snd` which gives the second element of a pair (tuple)
- **Implement each operation using Haskell**
 - `fst :: (a, b) -> a`
 - `fst :: (a, _) -> a`
 - `snd :: (a, b) -> b`
 - `snd :: (_, b) -> b`

40

40

Uses of Tuples

- **Tuples can be used to return multiple values from a function**
 - **Example:** `addAndSub :: Int -> Int -> (Int, Int)`
takes two integers and returns two integers (tuple) - first int returned is sum, second int returned is difference.
 - **Implement this in the Lab for practice**
- **Tuples can be used as function arguments**
 - **Example:** `add :: (Integer, Integer) -> Integer`
 - **Example:** `swap :: (a, b) -> (b, a)`

41

41

Sequences

- **Ordered Lists of elements**
- **Denoted by $\{a_n\}$, and terms of this sequence**
 a_i for i in $[0, 1, 2, \dots]$ or for i in $[1, 2, 3, \dots]$.
- **Example:** $a_n = (-1)^n$ is a sequence whose elements are $1, -1, 1, -1, 1, -1, \dots$, for i in $[0, 1, 2, \dots]$
- **Example:** $b_n = 2^n$ is a sequence whose elements are $2, 4, 8, 16, 32, \dots$, for i in $[1, 2, 3, \dots]$

42

42

Sequences

- Elements are of the same type
- Element x is at position i in the sequence s
- Sequences are defined using enumeration or comprehension - previous examples.

43

43

Sum of terms of a sequence

- Denoted by summation: - the sum of the terms in a sequence.
- Assume we have a sequence

$$a_1, a_2, a_3, \dots, a_n$$

We use the notation

$$\sum_{i=1}^n a_i$$

to represent

$$a_1 + a_2 + a_3 + \dots + a_n$$

44

44

Exercise

- Provide a recursive definition for sum of terms of a sequence.
 - Base case:
 - Recursive step:

45

45

Sum terms of a sequence

- Provide a simple specification for this operation

```
sumList :: [Int] -> Int
```

```
sumList [] = 0
```

```
sumList (x:xs) = x + sumList xs
```

- List other operations that could be performed on sequences

46

46

Provide recursive definition for below functions

1. To sum the first x elements of a sequence. Example, sum of first 5 terms.
2. To sum two sequences.
3. To find the difference between two sequences.
 - Provide specification for each above
 - Dry run
 - Implement in Haskell

47

47